TaskFlow:

A Collaborative Task Management Platform

By Miguel Angel de Pablo

TaskFlow:
A Collaborative Task Management Platform

INTRODUCTION

TaskFlow is a lightweight web application designed to help small teams stay organized without complex management tools. The core idea is simple: give students, freelancers, and small startups a focused space where they can create projects, assign tasks, and see updates happen in real time—without needing to constantly refresh the page or switch between apps.

The problem this solves is one I've experienced firsthand: when working on group projects, coordination often falls apart because lack of coordination. One person tracks tasks in a spreadsheet, another sends updates over chat, and important details get lost in the noise. TaskFlow brings those pieces together in a minimal, intentional interface that prioritizes clarity and immediate feedback. It's not trying to replace Jira or Asana; it's meant to be the tool you reach for when you need something that just works, easy and quickly.

PURPOSE AND AUDIENCE

Small teams consistently struggle with coordination because they rely on fragmented tools—spreadsheets for tracking, messaging apps for updates, and email for assignments. Information gets buried, responsibilities blur, and progress stalls. TaskFlow solves this by providing a single, focused workspace where teams can create projects, assign tasks, and track progress without unnecessary complexity. The application handles the full lifecycle of task management, from creation and assignment to status updates and completion, while keeping communication contextual to the work itself. Rather than adding another layer of administrative overhead, TaskFlow streamlines it, ensuring everyone knows exactly what needs to be done, who owns it, and when it’s due.

The intended users are small collaborative groups—typically three to ten people—who need structure without complexity. Think of a capstone team coordinating deliverables, a freelance designer and developer managing a client project, or a startup prototype squad tracking their sprint goals. These users don't need Gantt charts or resource allocation; they need to know who is doing what, by when, and what changed since they last looked. TaskFlow meets them where they are.

TECHNOLOGIES

Tool/Language	Application
React	Component-based frontend with hooks for state management
Python + Flask	RESTful backend API with blueprint architecture
PostgreSQL	Relational data modeling for users, projects, tasks
Axios	Typed API communication with error handling
JSON	Standardized request/response payloads
Git/GitHub	Version control, feature branches, PR workflow
CSS/HTML	Responsive, mobile-first styling with Flexbox/Grid

Methodology	Implementation
RESTful APIs	Resource-based endpoints (/api/tasks, /api/projects) with proper HTTP verbs
Object-Oriented Programming	Flask models and service classes for business logic
Control Structures & Algorithms	Task filtering, priority sorting, deadline validation logic
Agile/Project Management	2-week sprints, user stories, backlog prioritization
Code Quality	ESLint/Prettier (frontend), Flake8 (backend), meaningful commit messages
Problem Solving	Documented technical decisions in README and code comments

Beyond the core stack, I'm incorporating one feature I explored independently: real-time notifications using Socket.io. When a task is assigned, updated, or commented on, connected clients receive an immediate update without requiring a page refresh. This wasn't covered in the course curriculum, so I learned it through official documentation, community tutorials, and iterative testing. It cab adds genuine value to the collaborative experience. Importantly, this feature is implemented as a complement enhancement: if complexity becomes a blocker during development so we can reach the due date, the MVP can fall back to simple polling without losing core functionality.

TIMELIME

Total Estimated Duration: 7 weeks (part-time, aprox. 15 hrs/week)

Week	Milestone	Deliverable
1	**Project scaffolding & architecture**	Repo structure, Flask + React boilerplates, DB schema draft
2	**Backend core: Auth + CRUD APIs**	User registration/login, Task/Project endpoints with tests
3	**Frontend core: UI components + API integration**	Login flow, task list, create/edit forms, Axios service layer
4	**Collaboration features + real-time layer**	Project invitations, task assignment, Socket.io event wiring
5	**Responsive polish**	Mobile layout validation, cross-browser checks, error boundary handling
6	**Testing**	Final product test under several situations.
7	**Deployment + documentation**	Render deployment, environment config, final README, documentation